

An Exact Branch-Price-and-Cut Algorithm for the Unrelated Parallel Machine Scheduling Problem

Yang Yu,^{a,b} Xiaolong Li,^{a,b,*} Roberto Baldacci,^c Zhiqiao Wu,^{d,e,*} Wei Sun,^{d,*} Jiafu Tang,^d Han Zhu^d

^aSchool of Economics and Management, Dalian University of Technology, Dalian 116024, China; ^bNational Key Laboratory of Maritime Decision Intelligence, Dalian 116024, China; ^cCollege of Science and Engineering, Hamad Bin Khalifa University, Doha 34110, Qatar; ^dCollege of Management Science and Engineering, Dongbei University of Finance and Economics, Dalian 116025, China; ^eKey Laboratory of Liaoning Province for Data Analytics and Decision-Making Optimization, Dongbei University of Finance and Economics, Dalian 116025, China

*Corresponding authors

Contact: yuyangchen@dlut.edu.cn,  <https://orcid.org/0009-0003-7920-8209> (YY); luckydragon2023@163.com,  <https://orcid.org/0000-0001-7994-5581> (XL); rbaldacci@hbku.edu.qa,  <https://orcid.org/0000-0003-0938-5798> (RB); wuzhiqiao@dufe.edu.cn,  <https://orcid.org/0000-0002-7358-6175> (ZW); lnuhappysw@126.com,  <https://orcid.org/0000-0002-5176-8891> (WS); tangjiafu@mail.neu.edu.cn,  <https://orcid.org/0000-0002-0745-318X> (JT); hanzhu@dufe.edu.cn,  <https://orcid.org/0000-0002-5185-3324> (HZ)

Received: March 28, 2024

Revised: December 4, 2024; July 1, 2025;
February 11, 2026; April 30, 2026

Accepted: May 16, 2026

Published Online in Articles in Advance:
June 9, 2026

<https://doi.org/10.1287/ijoc.2024.0704>

Copyright: © 2026 INFORMS

Abstract. We investigate the Unrelated Parallel Machine Scheduling Problem (UPMSP), where independent jobs need scheduling on unrelated parallel machines. Each job has a distinct processing time, a weight, and a completion time. The goal is to find an optimal schedule that minimizes the total weighted completion time. This problem has been extensively studied in the scheduling literature. To address the UPMSP, we model feasible schedules using Smith's rule, representing them as paths on directed acyclic graphs, and derive a corresponding set-partitioning-based model. To solve the model, we introduce a branch-price-and-cut (BPC) algorithm with dynamic shrink upper bounds, incorporating state-of-the-art techniques such as the bucket graph-based labeling algorithm, limited-memory subset-row cuts, and strong branching. Notably, the proposed BPC algorithm is the first exact algorithm capable of addressing widespread real-life UPMSPs with rational-number processing times. Extensive numerical experiments demonstrate that our proposed algorithm outperforms state-of-the-art algorithms for the UPMSP with integer-valued processing times, solving larger-scale instances with up to 20 machines and 200 jobs efficiently. Furthermore, it effectively addresses real-life instances (from JD.com) with rational-number processing times involving up to 20 machines and 200 jobs.

History: Accepted by Andrea Lodi, Area Editor for Design & Analysis of Algorithms–Discrete.

Funding: Financial support from the National Natural Science Foundation of China [Grants 72571039, 72322018, 72272027, 72471050, and 72171043] and the Fundamental Research Funds for the Central Universities [Grant DUT23RC(3)070] is gratefully acknowledged.

Supplemental Material: The software that supports the findings of this study is available within the paper and its Supplemental Information (<https://pubsonline.informs.org/doi/suppl/10.1287/ijoc.2024.0704>) as well as from the IJOC GitHub software repository (<https://github.com/INFORMSJoC/2024.0704>). The complete IJOC Software and Data Repository is available at <https://informsjoc.github.io/>.

Keywords: machine scheduling • unrelated parallel machine • branch-price-and-cut • exact algorithm

1. Introduction

The Unrelated Parallel Machine Scheduling Problem (UPMSP) is a fundamental challenge in combinatorial optimization and has been extensively researched for decades. In practice, it has a wide range of applications, including production scheduling (Ekici et al. 2019, Iori et al. 2023), surgical scheduling (Sagnol et al. 2018, Zhao et al. 2026), cloud computing task scheduling (Chen et al. 2024), and combinatorial auctions for technician assignment (Yan et al. 2025), to name a few. Theoretically, this problem has motivated substantial research on developing effective solution methods. This trend is expected to continue in the coming years, highlighting the need to establish a foundation for future studies (Durasević and Jakobović 2023).

The UPMSP considered in this paper can be defined as follows. We are given a set N of n independent jobs that must be scheduled on a set M of m unrelated parallel machines. Let $N_k \subseteq N$, $N_k \neq \emptyset$ denote the set of jobs that can be processed on machine $k \in M$. To ensure the feasibility of the problem, we assume $\cup_{k \in M} N_k = N$. Each job $j \in N$ is associated with a weight w_j , where $w_j > 0$. Additionally, for each machine $k \in M$ and a job $j \in N_k$, a processing time $p_{jk} > 0$ is defined. Each machine can handle only one job from the set at a time, and each job must be

processed on a single machine without interruptions or preemption. Importantly, the processing times p_{jk} are *unrelated*—that is, they are arbitrary and do not possess any characteristic related to the machines.

Let C_j be the completion time of job $j \in N$ in a solution to the problem. The objective is to find a schedule that minimizes the total weighted completion time computed as $\sum_{j \in N} w_j C_j$. Based on the classification introduced by Lawler et al. (1993), the scheduling problem addressed in this paper is denoted $R \parallel \sum w_j C_j$; for simplicity, we refer to it as the UPMSP-WCT in the following.

The UPMSP-WCT has been proved to be $\mathcal{NP}$ -hard (Chen and Powell 1999). Because of the complexity of its exact solution, most studies on UPMSPs have focused on heuristic algorithms, such as Lin et al. (2011), Rodriguez et al. (2013), Xu et al. (2015), Rolim et al. (2023), and Maecker et al. (2023). Some works have explored exact methods—for instance, Chen and Powell (1999) proposed a branch-and-price (BP) algorithm, and Bulhões et al. (2020) designed a branch-price-and-cut (BPC) algorithm—but the scalability of these exact methods is limited, particularly concerning the number of jobs.

Additionally, with advancements in sensors and artificial intelligence, modern manufacturing systems can collect high-precision data, such as rational-number processing times, from various physical events to enable more accurate analysis, prediction, and control (van der Aalst 2016, Xu et al. 2020). Rational-number processing times are commonly observed in many real-world industrial applications, as demonstrated in studies such as Che et al. (2017), Ekici et al. (2019), and Iori et al. (2023). However, exact methods in the literature assume integer-valued processing times. When processing times involve rational numbers, scaling them to integer values can significantly slow performance, even for small-scale instances.

These observations motivated us to introduce a new exact method based on a BPC algorithm for the UPMSP-WCT. The new BPC algorithm can solve instances with twice the number of jobs as those handled by existing exact methods. Importantly, it achieves exact solutions for UPMSP-WCT instances with rational-number processing times for the first time.

Below, we provide a review of the literature related to the UPMSP-WCT, followed by a presentation of the contributions of this paper and an outline of its organization.

1.1. Related Work

The UPMSP has a broad spectrum of industrial applications (Maecker et al. 2023). Solving UPMSPs has prompted the development of various methods, including both heuristics and exact approaches. This section comprehensively reviews heuristic and exact methods for UPMSPs. For a review covering additional heuristics and exact methods for UPMSPs, the reader is referred to Durasević and Jakobović (2023).

1.1.1. Heuristic Methods. Numerous studies have focused on heuristic algorithms for UPMSPs and their variants. In their work, Lin et al. (2011) addressed UPMSPs with an emphasis on minimizing both total weighted completion time ($R \parallel \sum w_j C_j$) and total weighted tardiness ($R \parallel \sum w_j T_j$). They introduced the HEU-I heuristic for the former objective and two additional heuristics, HEU-II and ATC-I, for the latter. Another study by Rodriguez et al. (2013) addressed UPMSPs to minimize the total weighted completion time. They proposed an iterated greedy meta-heuristic (IGM) designed for large-scale instances.

Subsequent research has predominantly focused on exploring variants of the UPMSP, incorporating diverse practical considerations such as order splitting (Xu et al. 2015), the common due window (Rolim et al. 2023), and eligibility constraints (Maecker et al. 2023). Xu et al. (2015) investigated UPMSPs considering order splitting, aiming to minimize the total completion time ($R | O, \text{split} | \sum C_j$). They proposed three heuristics (H1, H2, and H3) for the problem.

Li et al. (2016) studied green UPMSPs that minimize the total completion time ($R | \text{Green} | \sum C_j$), taking into account energy costs and clean-up costs. They proposed a smallest-processing-time heuristic (SPTH) for the problem. Rolim et al. (2023) investigated just-in-time UPMSPs considering a common due window—that is, $R | r_j, d_j, \text{CDW} | \sum w'_j E_j + w_j T_j$. They introduced two mixed-integer linear programming (MILP) models and a novel earliness-tardiness extension of the adaptive large neighborhood search (ALNS) to address the problem.

Recently, Maecker et al. (2023) addressed UPMSPs considering job-machine-dependent delivery times (q_{ij}) and eligibility constraints (M_j), with the objective of total weighted tardiness minimization ($R | q_{ij}, M_j | \sum w_j T_j$). They presented a MILP model and implemented a variable neighborhood search (VNS) algorithm.

1.1.2. Exact Methods. Most UPMSPs and their variants have been proven to be $\mathcal{NP}$ -hard, presenting significant computational challenges. In their study, Azizoglu and Kirca (1999) focused on UPMSPs to minimize the total weighted completion time. They provided a lower bound and designed a branch-and-bound (BB) algorithm that efficiently solved instances with up to three machines and 20 jobs. Chen and Powell (1999) proposed a BP

algorithm for $R||\sum w_j C_j$, efficiently handling instances involving up to 20 machines and 100 jobs. No existing literature reports better performance than the BP algorithm for $R||\sum w_j C_j$. Subsequently, Shim and Kim (2007) considered UPMSPs to minimize total tardiness. They developed several dominance rules and lower bounds for the problem and proposed a BB algorithm that provides optimal solutions for instances with up to five machines and 20 jobs. Additionally, Detienne et al. (2011) addressed the UPMSP to minimize the weighted number of tardy jobs ($R||\sum w_j U_j$). They presented a MILP model. They solved it using a standard BB algorithm in the off-the-shelf XPRESS-MP solver, obtaining optimal solutions for instances with up to nine machines and 100 jobs.

Subsequent research has primarily explored variants of UPMSPs, accounting for diverse practical considerations. In their work, Pereira Lopes and de Carvalho (2007) addressed UPMSPs with sequence-dependent setup times (ST_j) to minimize the total weighted tardiness ($R|r_j, d_j, ST_j|\sum w_j T_j$). They introduced a BP algorithm with an innovative column-generation acceleration technique called primal box, which efficiently solves instances with 50 machines and 150 jobs. Bulhões et al. (2020) presented a generic BPC algorithm for UPMSPs and tailored it for the characteristics of UPMSP to minimize the total weighted earliness and tardiness ($R|r_j, d_j, ST_j|\sum w'_j E_j + w_j T_j$). The algorithm demonstrated efficiency in solving instances of ($R|r_j, d_j, ST_j|\sum w'_j E_j + w_j T_j$) with up to four machines and 120 jobs. Finally, Shahvari et al. (2022) investigated unrelated parallel machine batch scheduling problems with job conflicts, aiming to minimize the total weighted completion time and tardiness ($R|r_j, d_j, ST_j, B, C|\sum w'_j E_j + w_j T_j$). They formulated an efficient MILP model and proposed a BP algorithm, successfully addressing instances with up to six machines and 14 group jobs.

Table 1 provides an overview of heuristics and exact methods for UPMSPs and their variants.

However, existing exact methods have limitations in handling larger instances, especially with respect to the number of jobs. For instance, exact methods based on column generation for the $R||\sum w_j C_j$ typically solve instances with up to 100 jobs. Moreover, these exact methods are not directly applicable to real-life instances with rational-number processing times. Even if these rational numbers are rescaled to integers, this will lead to a hundredfold increase in the time horizon. As noted by van den Akker et al. (1999) and Bulhões et al. (2020), the efficiency of these exact algorithms deteriorates as the time horizon increases. Consequently, they become notably slow, failing to provide exact solutions within acceptable time frames, even for modest instances (Sadykov et al. 2021).

Table 1. Overview of UPMSP and Its Variants

Reference	Problem classification	Solution approach	Data set	Size
Lin et al. (2011)	$R \sum w_j C_j, R \sum w_j T_j$	Heuristic (HEU-I, HEU-II, ATC-I)	Random	$m = 10$ $n = 100$;
Rodriguez et al. (2013)	$R \sum w_j C_j$	Heuristic (IGM)	Random	$m = 50$ $n = 1,000$;
Xu et al. (2015)	$R O, \text{split} \sum C_j$	Heuristic (H1, H2, H3)	Random	$m = 20$ $n = 50$;
Li et al. (2016)	$R \text{Green} \sum C_j$	Heuristic (SPTH)	Random	$m = 5$ $n = 14$;
Rolim et al. (2023)	$R r_j, d_j, \text{CDW} \sum w'_j E_j + w_j T_j$	Heuristic (ALNS)	Random	$m = 6$ $n = 200$;
Maecker et al. (2023)	$R q_{ij}, M_j \sum w_j T_j$	Heuristic (VNS)	Random	$m = 20$ $n = 320$;
Azizoglu and Kirca (1999)	$R \sum w_j C_j$	Exact (BB)	Random	$m = 3$ $n = 20$;
Chen and Powell (1999)	$R \sum w_j C_j$	Exact (BP)	Random	$m = 20$ $n = 100$;
Shim and Kim (2007)	$R \sum w_j T_j$	Exact (BB)	Random	$m = 5$ $n = 20$;
Detienne et al. (2011)	$R \sum w_j U_j$	Exact (BB)	Random	$m = 9$ $n = 100$;
Pereira Lopes and de Carvalho (2007)	$R r_j, d_j, ST_j \sum w_j T_j$	Exact (BP)	Random	$m = 50$ $n = 150$;
Bulhões et al. (2020)	$R r_j, d_j, ST_j \sum w'_j E_j + w_j T_j$	Exact (BPC)	Şen and Bülbül (2015)	$m = 4$ $n = 120$;
Shahvari et al. (2022)	$R r_j, d_j, ST_j, B, C \sum w'_j E_j + w_j T_j$	Exact (BP)	Random	$m = 6$ $g = 14$;
This paper	$R \sum w_j C_j$ (UPMSP-WCT)	Exact (BPC)	Chen and Powell (1999)	$m = 20$ $n = 200$

Note. g , number of job families; m , number of machines; n , number of jobs.

1.2. Contributions of This Paper

This paper presents a new exact algorithm for the UPMSP-WCT. The contributions of this study are summarized as follows:

- We formulate the UPMSP-WCT using a set-partitioning-based model where we represent the feasible schedules of the problem as paths on m directed acyclic graphs, where m machines are involved and one graph is created for each machine, using Smith's rule (Smith 1956).
- The set-partitioning model is solved using a BPC algorithm that integrates several state-of-the-art techniques, including bucket graph-based labeling, subset-row cuts, dual price smoothing, reduced-cost fixing, and strong branching.
- To solve the pricing problem associated with the set-partitioning model, tailored to the characteristics of directed acyclic graphs, we propose a newly designed labeling algorithm, which can handle instances with rational-number processing times for the first time. We propose a dynamic shrink upper bound algorithm to compute upper bounds on job completion times, thereby improving the efficiency of the bucket graph-based labeling algorithm. As arcs are removed from directed acyclic graphs, these upper bounds shrink, helping prevent the creation of redundant buckets containing few or no labels and improving the efficiency of label dominance checks.
- We conduct extensive computational experiments, and the results show that the new BPC algorithm outperforms the current state-of-the-art algorithms. It efficiently addresses instances with integer and rational-valued processing times involving up to 20 machines and 200 jobs. Additionally, we aim to publish our data set and results, fostering accessibility for fellow researchers to encourage comparisons and further investigations.

The remainder of this paper is organized as follows. Section 2 represents feasible schedules of the UPMSP-WCT as paths on directed acyclic graphs according to Smith's rule (Smith 1956). This section also develops a mathematical formulation and valid inequalities for the UPMSP-WCT. Section 3 describes the pricing algorithm associated with the mathematical formulation. Section 4 presents the new BPC algorithm. Section 5 details the results of extensive computational experiments conducted on instances from the literature and real-world scenarios. Finally, Section 6 provides the conclusions of this study. Proofs of the statements and additional details about the BPC algorithm are available in the online supplement to this paper.

2. A Mathematical Formulation for the UPMSP-WCT and Its Relaxation

In this section, we first introduce the job-ordering restriction constructed following Smith's rule (Smith 1956). With these job-ordering restrictions, we represent feasible schedules of the UPMSP-WCT as paths in directed acyclic graphs. Furthermore, we adopt a set-partitioning formulation proposed by Chen and Powell (1999) for the UPMSP-WCT and use valid inequalities to enhance the linear relaxation of the model. Additionally, we describe the pricing problem associated with the formulation.

2.1. Representing Feasible Schedules

For any UPMSP, there may exist a predefined *job-ordering restriction* that specifies a set of jobs that must be scheduled before or after each job (Chen and Powell 1999). For UPMSP-WCT, its job-ordering restriction can be enforced internally via optimality properties—that is, the well-known Smith's rule (Smith 1956). In Smith's rule, the jobs processed on each machine must follow the Shortest Weighted Processing Time (SWPT) order in any optimal schedule—that is, if jobs i and j are both processed on machine k , with i preceding j , then the ratio of p_{ik} to w_i must be less than or equal to the ratio of p_{jk} to w_j .

We define a schedule for a machine $k \in M$ as a sequence $S = (j_1, j_2, \dots, j_p)$ of p jobs that satisfy the given job-ordering restriction such that $\{j_1, \dots, j_p\} \subseteq N_k$. A solution to the UPMSP-WCT can be represented by a set of schedules $\{S_q\}_{q \in \overline{M}}$, where $\overline{M} \subseteq M$ and S_q is the schedule associated with machine q . The solution is feasible if (i) each machine processes only one job at a time and (ii) each job is processed without pre-emption on exactly one machine. Let C_j be the completion time of job j —that is, the time the job terminates its operations on the assigned machine. The UPMSP-WCT seeks to find a solution S such that $\sum_{j=1}^n w_j C_j$ is minimized.

Let SWPT^k represent the SWPT order formed by all the jobs on machine k . Without loss of generality, we assume that if two jobs i and j have the same ratio—that is, $p_{ik}/w_i = p_{jk}/w_j$ —then the job with the smaller index precedes the other one in the SWPT^k sequence. Therefore, the job-ordering restriction for UPMSP-WCT can be represented as follows, for $j \in N_k$ and $k \in M$: $\alpha_j^k = \{i \in N_k : i \text{ succeeds } j \text{ in the sequence } \text{SWPT}^k\}$, $\beta_j^k = \{i \in N_k : i \text{ precedes } j \text{ in the sequence } \text{SWPT}^k\}$.

Based on the SWPT rule, a schedule for machine k can be represented as a directed acyclic graph, where each job corresponds to a vertex and the arcs represent the job-ordering restriction. More specifically, let $G^k = (V, A^k)$ be a directed graph for machine k , where V is the set of nodes and A^k is a set of arcs based on α_j^k and β_j^k . The node set V contains $|N_k| + 2$ vertices, and set V is partitioned as $V = \{0\} \cup N_k \cup \{n+1\}$. Vertex 0 represents the machine

start vertex, and vertex $n + 1$ represents the machine end vertex. The other vertices represent the jobs to be processed. Given these definitions, vertex 0 can precede any vertex in N_k , while vertex $n + 1$ can succeed any vertex in N_k . Consequently, the job-ordering restrictions are extended as follows: $\alpha_0^k = N_k \cup \{n + 1\}$, $\alpha_i^k = \alpha_i^k \cup \{n + 1\}$, $\beta_{n+1}^k = N_k \cup \{0\}$, $\beta_j^k = \beta_j^k \cup \{0\}$, for all $i, j \in N_k$. The arc set A^k consists of the arcs (i, j) , $\forall i \in N_k \cup \{0\}, j \in \alpha_i^k$ or $\forall j \in N_k \cup \{n + 1\}, i \in \beta_j^k$. A path P in the graph G^k is defined as a sequence of vertices $P = (j_0 = 0, j_1, \dots, j_{p-1}, j_p = n + 1)$, $j_1, \dots, j_{p-1} \in N_k$, $(j_{i_1}, j_{i_2}) \in A^k$, $i_1 = 0, \dots, p - 1$, $i_2 = i_1 + 1, \dots, p$. The path starts at vertex 0, visits a subset of vertices of N_k , and ends at vertex $n + 1$, corresponding to a feasible schedule on machine k . It is clear that SWPT^k is a total order on N_k . Therefore, the graph G^k , which represents the order relation, is acyclic due to its properties of antisymmetry and transitivity (Heath and Pemmaraju 1999).

2.2. A Set-Partitioning-Based Formulation

The set-partitioning model finds widespread application in addressing UPMSPs (Chen and Powell 1999, Bulhões et al. 2020), and in this section, we describe a set-partitioning-based model for the UPMSP-WCT.

Let Ω^k denote the index set of all feasible schedules for machine $k \in M$. Given $s \in \Omega^k$, we denote the corresponding schedule as $S_s = (j_0 = 0, j_1, \dots, j_{p-1}, j_p = n + 1)$, and we denote with C_{js}^k the completion times of jobs $j \in \{j_1, \dots, j_{p-1}\}$ in schedule S_s . The cost f_s^k of schedule S_s can be computed as $f_s^k = \sum_{j \in \{j_1, \dots, j_{p-1}\}} w_j C_{js}^k$. Let $a_{js}^k = 1$ be a 0-1 coefficient equal to one if schedule $s \in \Omega^k$ contains job j on machine k , and zero otherwise. Let y_s^k be a binary variable, where $k \in M$ and $s \in \Omega^k$, equal to one if schedule S_s is processed on machine $k \in M$, and zero otherwise. Formulation (SP) is as follows:

$$\min \sum_{k \in M} \sum_{s \in \Omega^k} f_s^k y_s^k, \quad (1)$$

$$\text{s.t.} \sum_{k \in M} \sum_{s \in \Omega^k} a_{js}^k y_s^k = 1, \quad \forall j \in N, \quad (2)$$

$$\sum_{s \in \Omega^k} y_s^k \leq 1, \quad \forall k \in M, \quad (3)$$

$$y_s^k \in \{0, 1\}, \quad \forall j \in N, k \in M. \quad (4)$$

The Objective Function (1) aims to find a set of feasible schedules with minimal cost. Constraints (2) and (3) ensure that each job is processed exactly once, and each machine is used at most once. Constraint (4) defines the domain of the decision variables.

2.2.1. Solving the Pricing Problem. Formulation SP involves an exponential number of decision variables, and its linear programming (LP) relaxation, LSP, can be effectively addressed via column generation. In each iteration of the column generation procedure, we solve the restricted master problem (RMP) defined by LSP, utilizing a limited set of decision variables. The resultant dual solution serves as input for the pricing algorithm, which solves the pricing problem. For a comprehensive overview of the column generation technique, we refer the reader to Barnhart et al. (1998) and Lübbecke and Desrosiers (2005).

The pricing problem associated with formulation LSP can be decomposed into m problems, one for each machine. As shown also by van den Akker et al. (1999), the pricing problem for machine $k \in M$ consists of finding a schedule or a path in graph G^k from vertex 0 to vertex $n + 1$ having negative reduced cost computed for the current dual solution (if any). The resulting optimization problem has been proven to be $\mathcal{NP}$ -hard (Chen and Powell 1999), and it can be conveniently solved in pseudo-polynomial time in $O(n^2 H)$ using dynamic programming, where $H = \sum_{j \in N} p_{jk}$ represents the length of the planning horizon. Nevertheless, dynamic programming approaches for this problem face computational challenges when handling instances with rational processing times. This is because these rational processing times need to be rescaled to integers, which can result in a significant increase in the time horizon H , potentially by a factor of hundreds, leading to longer running times for the dynamic programming method (van den Akker et al. 1999). Moreover, the effectiveness of the LSP relaxation can be compromised by its inherent weakness. Strengthening its lower bound values with additional cuts has proven to be an effective approach for problems modeled with set-partitioning-based formulations (Costa et al. 2019).

The next section presents the additional cuts for formulation LSP, while Section 3 describes the strengthened pricing algorithm.

2.3. Additional Cuts

Robust and nonrobust cuts are commonly employed in vehicle routing problems (Jepsen et al. 2008; Pecin et al. 2017a, b; Sadykov et al. 2021). Robust cuts, typically formulated using arc-flow-indexed variables, preserve the structure of the pricing problem (Fukasawa et al. 2006). Although effective in closing the integrality gap for some easy instances, they are insufficient for harder ones (Costa et al. 2019). To address these more challenging instances, this section employs nonrobust cuts.

Subset-row cuts (SRCs) are widely recognized as an effective class of nonrobust cuts (Pecin et al. 2017a, b). Following Jepsen et al. (2008), the application of the Chvátal-Gomory rounding procedure to Constraint (2) yields a family of SRCs for the UPMSPP-WCT. Specifically, for a given base set $C \subseteq N$ and a multiplier ρ where $0 < \rho < 1$, we define the SRCs as follows:

$$\sum_{k \in M} \sum_{s \in \Omega^k} \left\lfloor \rho \sum_{j \in C} a_{js}^k \right\rfloor y_s^k \leq \lfloor \rho |C| \rfloor. \quad (5)$$

Here, $\rho = \frac{1}{q}$ with $q \in \{1, 2, \dots, |C|\}$. By varying the cardinality of C and the value of ρ , different families of SRCs can be derived. Because of the floor operator ($\lfloor \cdot \rfloor$), the coefficient of a variable y_s^k in an SRC is not a linear expression of arc-flow-indexed variables, thus classifying these cuts as nonrobust (Sadykov et al. 2021). Incorporating such nonrobust cuts increases the complexity of the pricing problem, because it requires additional unrestricted resources to handle the associated dual contribution.

To mitigate the impact on labeling algorithms, we use the limited-memory SRCs (lm-SRCs) proposed by Pecin et al. (2017a) instead of SRCs. The lm-SRCs are defined as follows:

$$\sum_{k \in M} \sum_{s \in \Omega^k} \omega(C, \mathcal{M}^k, \rho, s, k) y_s^k \leq \lfloor \rho |C| \rfloor. \quad (6)$$

The lm-SRC is associated with a machine index k , a base set $C \subseteq N$, a multiplier ρ , a coefficient function ω , and a memory set $\mathcal{M}^k$ with $C \subseteq \mathcal{M}^k \subseteq N$. The coefficient ω is computed using an auxiliary variable referred to as *state*. Each time a vertex in C is visited, the variable *state* is increased by ρ . When *state* becomes greater than or equal to $\lfloor \rho |C| \rfloor$, its value is reduced by $\lfloor \rho |C| \rfloor$, and ω is incremented. For the sake of space, the procedure for computing ω and additional information about the memory set $\mathcal{M}^k$ are described in Online Supplement EC.2.1. In our implementation, we adopt the widely used parameters $|C| = 3$ and $\rho = \frac{1}{2}$. For more details on lm-SRCs, we direct the reader to Pecin et al. (2017a, b).

3. Solving the Pricing Problem: A Bucket Graph-Based Labeling Algorithm with Dynamic Shrink Upper Bounds

This section outlines the labeling algorithm for solving the pricing problem associated with machine $k \in M$, denoted PSP^k . Initially, labels are defined for PSP^k that account for nonrobust cuts. Then, extension and dominance rules applicable to these labels are introduced (Section 3.1). Specifically, the problem-specific bucket graph is designed to handle rational-number processing times while avoiding the discretization of the time horizon. To enhance the labeling algorithm's efficiency, we propose the dynamic shrink upper bound algorithm, which capitalizes on the problem's specific characteristics (Section 3.2 and Section 3.3). The forward labeling algorithm is introduced in Section 3.4, followed by a description of the pricing strategy (Section 3.5), including heuristic pricing strategies.

Let $\pi_j \in \mathbb{R}$, $\forall j \in N$, and $\sigma^k \leq 0$, $\forall k \in M$, be the dual variables associated with Constraints (2) and (3), respectively. Additionally, let $\mathcal{C}$ be the index set of the lm-SRCs, and let $\mu_l \leq 0$, $\forall l \in \mathcal{C}$ be the dual variables associated with the lm-SRCs. The reduced cost of schedule $S_s, s \in \Omega^k, k \in M$ is

$$r_s^k = f_s^k - \sum_{j \in N} a_{js}^k \pi_j - \sigma^k - \sum_{l \in \mathcal{C}} \omega(C, \mathcal{M}^k, \rho, s, k) \mu_l. \quad (7)$$

The pricing problem associated with machine $k \in M$ asks for a schedule $S_{s'}, s' \in \Omega^k$, of minimum reduced cost. If $r_{s'}^k < 0$, the schedule is added to the RMP; otherwise, the solution (π, σ, μ) is proven to be a feasible dual solution to the LSP with the additional cuts.

Hereafter, we omit the index k for clarity in the description, because PSP^k is the same for each machine and we use only the p_{jk} values. We define $\hat{p}_j$ as the generic processing time for job j , where $\hat{p}_j = p_{jk}$.

3.1. Label Definition, Extension, and Dominance Rules

Unlike the PSPs in Chen and Powell (1999) and Wang et al. (2018), our PSP is more challenging due to the inclusion of dual contributions from additional (nonrobust) cuts.

We formulate the PSP on a graph $G = (V, A)$, which is the same directed acyclic graph described in Section 2.1. Let $r_a, a \in A$ be the *reduced cost* of arc a ; the time consumption of each arc (j, j') is equal to $\hat{p}_{j'}$. A path P on graph G is represented by its arcs: $P = (a_1^P, \dots, a_{|P|}^P)$, where $a_e^P = (j_{e-1}^P, j_e^P)$, $e = 1, \dots, |P|$, with $j_0^P = 0$, and $j_{|P|}^P = n + 1$. The path P yields a completion time C_{j_h} at vertices, which can be computed as $C_{j_h} = 0$, if $h = 0$; otherwise, $C_{j_{h-1}} + \hat{p}_{j_h}$. The reduced cost of path P can be rewritten as

$$r_P = \sum_{a \in P} r_a - \sum_{l \in \mathcal{C}} \omega(C, \mathcal{M}, \rho, P) \mu_l, \quad (8)$$

where $r_a = f_a - \pi_a$, $f_a = w_{j'}$, $w_0 = w_{n+1} = 0$, and $\pi_a = \pi_{j'}$, $\pi_0 = \sigma$, $\pi_{n+1} = 0$ for $a = (j, j') \in A$.

We refer to a tuple $L = (v, \mathcal{V}, C, r, \mathcal{S}, \alpha)$, representing a partial schedule as a path P in the graph G , as a forward *label*. In this tuple, v denotes the last job in the schedule; $\mathcal{V}$ represents the set of jobs, including job v , contained in the schedule; C represents the completion time of the current schedule; r is the reduced cost of the schedule; $\mathcal{S}$ is the vector of *states* corresponding to n_S lm-SRCs, as described in Section 2.3; and α represents the job-ordering restriction—that is, the set of jobs that can be scheduled after the last job. An initial forward label is denoted by $L^{init} = (0, \emptyset, 0, 0, \emptyset, \{1, \dots, n\})$. In the following, we denote the component x of label L by $x(L)$.

Let L' be the forward label obtained by extending forward label L along arc $(v(L), j)$. It is essential to note that vertex j must belong to the set $\alpha(L)$. The extension rule for L is defined as follows: (i) $v(L') = j \in \alpha(L)$; (ii) $\mathcal{V}(L') = \mathcal{V}(L) \cup \{j\}$; (iii) $C(L') = C(L) + \hat{p}_j$; (iv) $r(L') = r(L) + r_{(v(L), j)}$; and (v) $\alpha(L') = \alpha_j$. The computation of the vector $\mathcal{S}(L')$ and the update of the reduced cost $r(L')$ based on the lm-SRCs are outlined in Algorithm 1. A forward label L is complete if it includes the end vertex $n + 1$.

Algorithm 1 (Calculate Vector $\mathcal{S}(L')$)

Input: $\mathcal{S}(L), r(L'), \mu, \mathcal{M}, \rho$

Output: $\mathcal{S}(L'), r(L')$

begin

```

1   $\mathcal{S}(L') \leftarrow \mathcal{S}(L)$ ;
2  for  $l = 0$  to  $n_S$  do
3    if  $j \notin \mathcal{M}(l)$  then
4       $\mathcal{S}(L')[l] \leftarrow 0$ ;
5    else if  $j \in \mathcal{C}(l)$  then
6       $\mathcal{S}(L')[l] \leftarrow \mathcal{S}(L')[l] + \rho$ ;
7      if  $\mathcal{S}(L')[l] \geq 1$  then
8         $r(L') \leftarrow r(L') - \mu_l$ ,  $\mathcal{S}(L')[l] \leftarrow \mathcal{S}(L')[l] - 1$ ;
9      end
10 end
11 return  $\mathcal{S}(L'), r(L')$ ;
12 end
```

To avoid enumerating all feasible labels, dominance rules are applied to identify and discard dominated labels. A label L_1 is said to dominate a label L_2 if every feasible completion of L_2 is also feasible for L_1 and yields a label whose reduced cost is not greater than that of the label obtained by applying the same completion to L_1 . The following proposition holds.

Proposition 1. Let L_1 and L_2 be two forward labels with $v(L_1) = v(L_2)$. If $C(L_1) \leq C(L_2)$ and

$$r(L_1) - \sum_{l \in \mathcal{C}: \mathcal{S}(L_1)[l] > \mathcal{S}(L_2)[l]} \mu_l \leq r(L_2), \quad (9)$$

with at least one of these inequalities is strict, then label L_1 dominates label L_2 , and thus label L_2 can be discarded.

In the above proposition, the second term on the left-hand side of Inequality (9) represents a lower bound on the additional dual contribution a completion of L_1 may incur compared with the same completion of L_2 , if it revisits

sets $C(l)$ for l -SRC $l \in C$, where $S(L_1)[l] > S(L_2)[l]$. Dominance can only occur if the reduced cost of L_1 is sufficiently lower than that of L_2 to compensate for the potential additional penalization because $\sum_{l \in C: S(L_1)[l] > S(L_2)[l]} \mu_l \leq 0$. Only nondominated labels are retained.

3.2. Dynamic Shrink Upper Bounds Algorithm: Initializing and Updating Upper Bounds on the Completion Times of the Jobs

In the directed acyclic graph corresponding to PSP^k , for a job j_h on machine k , we introduce lower and upper bounds on the completion time— $lb_{j_h}^k$ and $ub_{j_h}^k$. Initially, we set $lb_{j_h}^k$ to $p_{j_h k}$ and $ub_{j_h}^k$ to $\sum_{j \in N_k} p_{jk}$. The schedule is considered *feasible* if all jobs contained within it satisfy the specified time interval—that is, $lb_{j_h}^k \leq C_{j_h}^k \leq ub_{j_h}^k$ for $j_h \in S_s, s \in \Omega^k$. A tight time interval is necessary to eliminate many infeasible schedules. Promoting this property to the labeling algorithm, this tight time interval can improve the efficiency of the pricing algorithm, similar to the approach in Kowalczyk and Leus (2018). In our study, however, this tight time interval mitigates the drawbacks of the bucket graph for the directed acyclic graph corresponding to PSP^k , improving the efficiency of label dominance checks within buckets. The specific reasons for this will be explained later. Below, we describe how to obtain tight upper bounds and continuously shrink them.

Given the characteristics of the directed acyclic graph, we propose a *dynamic shrink upper bound algorithm* that takes $ub_{j_h}^k$ as input and returns the updated $ub_{j_h}^k$. It is termed *dynamic shrink* because the upper bound may shrink once an arc is removed from the directed acyclic graph. In this section, we first present a recurrence equation for updating the upper bound. Subsequently, we prove the accuracy of the updated upper bound. Finally, we describe the dynamic shrink upper bounds algorithm in detail.

Let $(j_0 = 0, j_1, \dots, j_n, j_{n+1} = n + 1)$ denote the set of all jobs arranged in $SWPT^k$ order on machine k . When considering a job j_h along with its job-ordering restriction $\beta_{j_h}^k$, we can compute an upper bound on the job completion time on machine k using the following recurrence method:

$$ub_{j_h}^k = \min \left\{ ub^*, \max_{j' \in \beta_{j_h}^k} \{ ub_{j'}^k \} + p_{j_h k} \right\}, \quad (10)$$

computed for $j_h = j_1, \dots, j_n, j_{n+1}$, where $ub_{j_0}^k$ is set to zero and ub^* is an upper bound for the completion time on any machine, computed as $ub^* = (\sum_{j \in N} \max_{k \in M} \{ p_{jk} \}) / m + \max_{k \in M, j \in N} \{ p_{jk} \}$. Based on the recurrence method, it is evident that the value of $ub_{j_h}^k$ depends on the job-ordering restriction $\beta_{j_h}^k$. This upper bound may shrink as some jobs are removed from $\beta_{j_h}^k$.

The following Theorem 1 shows the validity of the recurrence method.

Theorem 1. *An optimal solution exists for the completion time $C_{j_h}^k$ of job j_h on machine k such that $C_{j_h}^k \leq \min\{ub^*, \max_{j' \in \beta_{j_h}^k} \{ ub_{j'}^k \} + p_{j_h k}\}$.*

The following theorem introduces conditions under which removing arcs updates the upper bound.

Theorem 2. *For a given vertex j_h , its dynamic shrink upper bound $ub_{j_h}^k$ on machine k can be updated if the arc (j_{h-1}, j_h) with $j_{h-1} = \arg\max_{j' \in \beta_{j_h}^k} \{ ub_{j'}^k \}$ is removed from the directed acyclic graph G^k .*

With Theorem 2 and the Recurrence Relation (10), the dynamic shrink upper bounds algorithm is presented in Algorithm 2. Two auxiliary variables are introduced: $\maxPreJ_{j_h}^k = \arg\max_{j' \in \beta_{j_h}^k} ub_{j'}^k$, which identifies the job with the highest upper bound preceding job j_h on machine k ; and $U_{j_h}^k$, which indicates whether job j_h meets the conditions on machine k outlined in Theorem 2. Initially, within the algorithm's input, $\mathcal{A}^k$ stores arcs recently removed from the directed acyclic graph G^k . For initializing the upper bounds, $U_{j_h}^k$ is set as true, $\forall j_h \in \{j_0, \dots, j_n, j_{n+1}\}$. Subsequently, U^k is updated based on the removed arcs $\mathcal{A}^k$, as specified in lines 1–5 of the algorithm. Finally, identifying the job that satisfies Theorem 2 with the highest $SWPT^k$ order, as outlined in lines 6–10. Starting with this job, the upper bounds are calculated in $SWPT^k$ order, as shown in lines 11–22. This calculation corresponds to the previously given Recurrence Relation (10).

Algorithm 2 (Dynamic Shrink Upper Bounds Algorithm)

```

Input:  $ub^k, \mathcal{A}^k, \maxPreJ^k, \mathcal{U}^k$ 
Output:  $ub^k, \mathcal{A}^k, \maxPreJ^k, \mathcal{U}^k$ 
begin
1   for each arc  $a = (j_{h-1}, j_h)$  in  $\mathcal{A}^k$  do
2       if  $\maxPreJ_{j_h}^k = j_{h-1}$  then
3            $\mathcal{U}_{j_h}^k \leftarrow \text{true};$ 
4       end
5   end
6    $\mathcal{A}^k \leftarrow \emptyset, \text{isActive} \leftarrow \text{false};$ 
7   for each job  $j$  in  $\{j_0, \dots, j_n, j_{n+1}\}$  in SWPTk order do
8       if  $\mathcal{U}_j^k = \text{true}$  then
9            $\text{isActive} \leftarrow \text{true};$ 
10      end
11      if  $\text{isActive} = \text{true}$  then
12           $\mathcal{U}_j^k \leftarrow \text{false}, ub_j^k \leftarrow 0, \maxPreJ_j^k \leftarrow 0;$ 
13          for each job  $j'$  in  $\beta_j^k$  do
14              if  $ub_j^k \leq ub_{j'}^k$  then
15                   $ub_j^k \leftarrow ub_{j'}^k, \maxPreJ_j^k \leftarrow j';$ 
16              end
17          end
18           $ub_j^k \leftarrow ub_j^k + p_{jk};$ 
19          if  $ub_j^k > ub^*$  then
20               $ub_j^k \leftarrow ub^*;$ 
21          end
22      end
23  end
24  return  $ub^k, \mathcal{A}^k, \maxPreJ^k, \mathcal{U}^k;$ 
25 end

```

3.3. Bucket Graph with Dynamic Shrink Upper Bounds

Determining when to perform dominance checks between labels is a key challenge for the labeling algorithm. It is crucial to balance the efficiency and frequency of these checks: performing dominance checks for all label pairs can be prohibitively time-consuming, whereas skipping too many may lead to a premature explosion in the number of maintained labels.

To address this issue, Pecin et al. (2017b) partitioned the labels into buckets and performed dominance checks within each bucket. Bulhões et al. (2020) followed this approach to tackle the PSP of the UPMSP by defining a bucket for each job and every possible completion time, resulting in $O(nH)$ buckets, where H denotes the time horizon, and skipping interbucket dominance checks. However, as noted by Sadykov et al. (2021), the algorithm becomes slow when processing instances with arbitrary rational processing times. This is because these rational processing times must be rescaled to integers, which can significantly increase the time horizon H , leading to more buckets and fewer dominance checks. Consequently, the algorithm slows down because too many undetected dominated labels are expanded. In response to this challenge, we employ a bucket graph-based labeling algorithm for the PSP based on the one proposed by Sadykov et al. (2021) for routing problems, which conducts dominance checks with exceptional effectiveness and efficiency.

The proposed algorithm categorizes labels into buckets based on the last-visited job and its completion time range, thereby avoiding discretization of the time horizon. With this setup, rational-number processing times do not need to be rescaled to integers, with $O(\sum_{j \in N} R_j)$ buckets, where R_j is a predefined parameter indicating the number of buckets for job j . To further reduce undetected labels, it applies interbucket dominance checks and extends labels via bucket arcs.

The bucket graph for the PSP exhibits unique characteristics. When arcs are removed from the directed acyclic graph, upper bounds on the completion time of jobs shrink accordingly, as shown in Section 3.2. This reduction renders buckets beyond these updated upper bounds and their corresponding arcs redundant, because no labels extend into them. Consequently, the bucket graph's effectiveness in facilitating efficient label dominance checks within buckets may be reduced. However, by incorporating dynamic shrink upper bounds into the bucket graph, the efficiency of dominance checks can be enhanced.

For ease of description, we define the period between two dynamic shrink upper bound updates as one *iteration*. The bucket graph with dynamic shrink bounds is constructed for the t -th iteration. For simplification, the index k is omitted, and $\hat{p}_j = p_{jk}$ is defined as the generic processing time for job j . For each job $j \in N$, the corresponding buckets set is defined by three parameters: a step size d_j , a lower bound lb_j , and an upper bound $dsub_j$

updated by the dynamic shrink upper bounds algorithm. Define $I_j = \{0, 1, \dots, R_j - 1\}$, $j \in N$, as the index set for the buckets for job j . Consequently, $d_j(t) = (ds_{ub_j}(t) - lb_j)/R_j$. Define B as the set of all buckets for a single machine. Each bucket b is represented by a pair of job and lower bound (j_b, lb_b) , where $j_b = j$ and $lb_b = lb_j + i \cdot d_j(t)$, with $j \in N$ and $i \in I_j$. A forward label L is contained in bucket b if $v(L) = j_b$ and $lb_b \leq C(L) < lb_b + d_{j_b}(t)$.

Let Γ be the set of *bucket arcs*. Each bucket arc $\gamma \in \Gamma$ is defined by a tuple $(b_{\gamma}^{\text{tail}}, a_{\gamma}, b_{\gamma}^{\text{head}})$ of two buckets and an arc such that $b_{\gamma}^{\text{tail}}, b_{\gamma}^{\text{head}} \in B$, $a_{\gamma} \in A$. Specifically, bucket b_{γ}^{tail} represents the tail bucket, while bucket b_{γ}^{head} represents the head bucket of bucket arc γ . Set Γ contains a bucket arc if two buckets satisfy the following condition: $lb_{b_{\gamma}^{\text{head}}} \leq lb_{b_{\gamma}^{\text{tail}}} + \hat{p}_{j_{b_{\gamma}^{\text{head}}}} < lb_{b_{\gamma}^{\text{head}}} + d_{j_{b_{\gamma}^{\text{head}}}}(t)$. In addition to bucket arcs between different jobs, bucket arcs sharing the same job are also considered. The bucket graph introduces the concept of “component-wise smaller,” meaning that bucket b' is component-wise (CW) smaller than bucket b if $v_b = v_{b'}$ and $i_{b'} \leq i_b$. By this definition, we obtain the set $\Phi_b = \{b' : v_b = v_{b'}, i_{b'} = i_b - 1\}$ of “adjacent” CW smaller buckets for bucket b . Thus, we can derive the set Ψ of bucket arcs (b, b') , where $b \in B$ and $b' \in \Phi_b$.

3.4. Forward Labeling Algorithm

Our implementation of the forward labeling algorithm for the PSP is based on an adaptation of the algorithm proposed by Sadykov et al. (2021) for directed acyclic graphs.

The algorithm, detailed in Algorithm 3, involves four key changes: (i) It incorporates a dynamic shrink upper bounds algorithm that can reconstruct the bucket graph if necessary (lines 1–4), thereby improving the efficiency of dominance checks between labels within buckets; (ii) it eliminates the need to generate strongly connected components, reducing computation time. Processing buckets in SWPT and ascending index order avoids unnecessary extensions from processed buckets (line 7). In the original algorithm, strongly connected components serve the same purpose. (iii) It simplifies the dominance check between label L and labels in all CW smaller buckets (line 9). Instead of checking all CW smaller buckets, it processes them in descending order of their indices, using the smallest reduced cost in bucket b , r_b^{best} , to terminate the search early. If a CW smaller b' is found such that $r_{b'}^{\text{best}} > r(L)$, no label in b' or any smaller CW bucket can dominate L . (iv) It redefines the *Label Extension* function to extend a label L' to L along a bucket arc based on specific rules (in line 11). The function first updates the last visited job $v(L)$ for the new label L and verifies its feasibility with the job-ordering restriction $\alpha(L')$. It then computes the reduced cost $r(L)$ and completion time $C(L)$, ensuring that the completion time falls within the time interval $[lb_{v(L)}, ds_{ub_{v(L)}}]$. Finally, it updates the reduced cost according to the lm-SRCs using Algorithm 1. For more details, readers can refer to Sadykov et al. (2021).

Algorithm 3 (Forward Labeling Algorithm)

Input: $ub, A, \text{maxPreJ}, \mathcal{U}, B, \Gamma \cup \Psi, \alpha, \mu, \mathcal{M}, \rho$

Output: S^{best}

begin

```

1  Run Algorithm 2, Dynamic Shrink Upper Bounds Algorithm ( $ub, A, \text{maxPreJ}, \mathcal{U}$ ) to update  $ub$ ;
2  if  $ub$  updated then
3    | Reconstruct the bucket graph  $(B, \Gamma \cup \Psi)$  based on the updated  $ub$ ;
4  end
5   $L^{\text{init}} \leftarrow (0, \emptyset, 0, 0, \emptyset, \{1, \dots, n\})$ ;
6  Insert initial label  $L^{\text{init}}$  into its bucket  $b^{L^{\text{init}}}$  and mark  $L^{\text{init}}$  as nonextended;
7  for each bucket  $b \in B$  in SWPT order and in ascending order of bucket indices do
8    for each nonextended label  $L'$  where  $b^{L'} = b$  do
9      if label  $L'$  is not dominated by a label in bucket  $b'$  where  $j_{b'} = j_b$  from  $i_{b'} = i_b - 1$  to 0 then
10       for each bucket arc  $\gamma \in \Gamma$  where  $b_{\gamma} = b^{L'}$  do
11         if Label Extension( $L', \gamma, L, \alpha, \mu, \mathcal{M}, \rho$ ) (see Algorithm 4) then
12           if  $L$  is not dominated by a label in  $b^L$  then
13             | Mark  $L$  as nonextended and insert into  $b^L$ ;
14             | Remove labels dominated by  $L$  from  $b^L$ ;
15           end
16         end
17       end
18     end
19     Mark  $L'$  as extended;
20   end
21    $r_b^{\text{best}} \leftarrow \min\{\min_{L \text{ where } b^L=b}\{r(L)\}, \min_{b' \in \Psi_b}\{r_{b'}^{\text{best}}\}\}$ ;
22 end
23  $S^{\text{best}} = \text{argmin}_{S^L \text{ where } v(L)=n+1} r(L)$ ;
24 return  $S^{\text{best}}$ ;
25 end
```

Algorithm 4 (Label Extension)

Input: $L', \gamma, L, \alpha, \mu, \mathcal{M}, \rho$

Output: true or false

begin

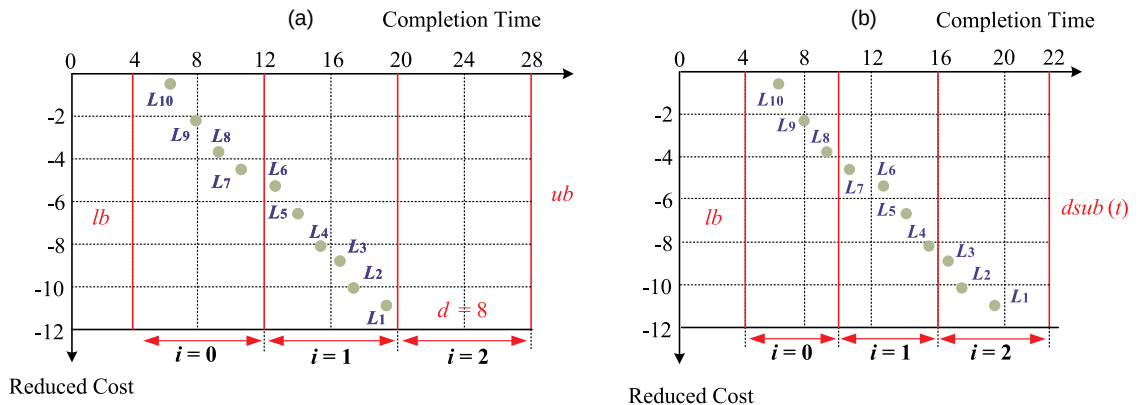
```

1   $v(L) \leftarrow \text{head of } a_\gamma;$ 
2  if  $v(L) \in \alpha(L')$  then
3     $\alpha(L) \leftarrow \alpha_{v(L)}^k, C(L) \leftarrow C(L'), r(L) \leftarrow r(L');$ 
4     $C(L) \leftarrow \max\{C(L) + p_{v(L)k}, lb_{v(L)}\};$ 
5    if  $C(L) > dsub_{v(L)}$  or  $C(L) < lb_{v(L)}$  then
6      return false;
7    end
8     $\mathcal{V} \leftarrow \mathcal{V} \cup \{v(L)\}, r(L) \leftarrow r(L) + r_{a_\gamma};$ 
9     $S(L), r(L) \leftarrow \text{CalculateVectorS}(L), \text{ using Algorithm 1 } (S(L'), r(L), \mu, \mathcal{M}, \rho);$ 
10 end
11 else
12   return false;
13 end
14 return true;
15 end
```

We now explain how dynamic shrink upper bounds improve the efficiency of dominance checks between labels within a job's buckets. The processing time for each bucket increases quadratically with the number of nondominated labels it contains (Pecin et al. 2017b). Let $T_i(n_i) = zn_i^2$ represent the time required for label dominance checks within bucket i , where i is the index of the bucket, n_i denotes the number of nondominated labels, and z is a constant reflecting the time complexity or other processing factors. For any job j , the total time for label dominance checks in the corresponding buckets is $T = z \sum_{i=0}^{R_j-1} n_i^2$. By the convexity principle, for a fixed number of buckets and nondominated labels, this time T is minimized when labels are distributed uniformly. This implies that allocations where some buckets are empty are generally inefficient. Because the label extension follows the job-ordering restriction α , the extended labels naturally do not exceed the dynamic shrink upper bounds. Therefore, for the original bucket graph proposed by Sadykov et al. (2021), many redundant buckets beyond dynamic shrink upper bounds, containing few or no labels, may exist, significantly impacting the efficiency of dominance checks. The labeling algorithm typically runs hundreds of iterations, making this inefficiency substantial.

By reconstructing the bucket graph using tighter dynamic shrink upper bounds, the uneven distribution of labels—where some buckets are empty whereas others contain a high concentration—can be effectively mitigated, thereby improving the efficiency of dominance checks. This effect is illustrated in Figure 1, where panel (a) shows buckets for a job without a dynamic shrink upper bound, and panel (b) shows ones with this bound at iteration t . Both subfigures plot reduced cost (on the y -axis) against completion time (on the x -axis) and include three buckets ($i = 0, i = 1$, and $i = 2$) along the x -axis. These buckets contain forward labels ranging from L_1 to L_{10} , and the labels within each bucket are nondominated. Each forward label is positioned according to its

Figure 1. (Color online) Buckets Without Dynamic Shrink Upper Bound (a) vs. Buckets with Dynamic Shrink Upper Bound (b)



completion time and reduced cost, and each bucket contains some forward labels within the specified bounds. The two subfigures share the same lower bound, $lb = 4$, but differ in that panel (a) uses a fixed upper bound of $ub = 28$, whereas panel (b) employs a dynamic shrink upper bound of $dsub(t) = 22$, resulting in different bucket step sizes—that is, $d = 8$ and $d(t) = 6$. Consequently, as shown in panel (a), there are no labels in bucket $i = 2$, with most labels (L_1 – L_6 , six forward labels) concentrated in bucket $i = 1$. The remaining labels, L_7 – L_{10} , are contained in bucket $i = 0$. The time spent on dominance checks can be approximately calculated as $T_a = 52z$. In contrast, panel (b) shows a more balanced distribution of labels across buckets, avoiding empty buckets—that is, labels L_1 – L_3 are in bucket $i = 2$, L_4 – L_7 are in bucket $i = 1$, and L_8 – L_{10} are in bucket $i = 0$. The time spent on dominance checks can be approximately calculated as $T_b = 34z$. Thus, about 34% of the computation time is reduced in this example.

3.5. Pricing Strategy

Constructing an exact backward labeling algorithm for the PSP is nontrivial and presents challenges distinct from those of the forward labeling algorithm. Unlike the forward approach, which is anchored at a fixed start time $t = 0$, the backward algorithm initializes at the planning horizon H . A critical difficulty arises because the actual start time of any partial schedule is constrained by the completion time of the preceding job, which is unknown during the backward extension. This will affect the total weighted completion time and, consequently, the reduced cost. Therefore, we cannot simply store a scalar value for the reduced cost; instead, it must be formulated as a function of the start time t .

To address this, we design an exact backward labeling algorithm that leverages the affine structure of the reduced cost (detailed in Online Supplement EC.2.3). However, this affine function yields strictly weaker dominance rules (see Proposition EC.2 in the online supplement) than the forward labeling algorithm. This results in a significantly larger set of nondominated labels, increasing the frequency of dominance checks and slowing the algorithm per iteration. Given this computational overhead, we rely solely on the exact forward labeling algorithm to solve the PSP, reserving the exact backward labeling algorithm exclusively for the variable fixing approach (see Section 4.3.2 and Online Supplement EC.2.4).

The exact (forward labeling) pricing algorithm still requires significant computation time. Consequently, faster heuristic pricing methods may expedite the column-and-cut generation process. The exact pricing method becomes essential only when the heuristics fail to identify schedules with negative reduced costs. In our implementation, we execute the neighborhood search heuristic pricing algorithm. This method is based on the forward labeling algorithm but operates heuristically. The label extension of exact pricing is adapted to operate heuristically. Additional details about the heuristic pricing algorithm can be found in Online Supplement EC.2.2, including the specific procedure outlined in Algorithm 8 of the Online Supplement, along with related descriptions.

4. A BPC Algorithm for the UPMSP-WCT

This section describes the BPC algorithm for solving the UPMSP-WCT. Our implementation is built upon column-and-cut generation, branch-and-bound, and state-of-the-art acceleration techniques. Below, we outline the main components of our BPC algorithm. The recent publications by Desrosiers et al. (2024) and Uchoa et al. (2024) provide an up-to-date and thorough review of BPC algorithms and related techniques.

4.1. Column-and-Cut Generation

The column-and-cut generation method, which combines column and cut generation, is widely employed to address large-scale problems (see, e.g., Costa et al. 2019).

At each node of the branch-and-bound tree, the process begins by solving the linear relaxation of the RMP with lm-SRCs and branching constraints. Before each cut generation iteration, the current relaxation of the set-partitioning formulation is solved via column generation. We employed the automatic dual pricing smoothing stabilization method in our implementation, as described by Pessoa et al. (2018). At each column-generation iteration, the primal simplex algorithm is used to solve the RMP and obtain primal and dual solutions. Subsequently, each PSP, denoted PSP^k , is solved to identify columns with negative reduced costs. If no columns with negative reduced costs are found for any PSP^k , the current primal solution is considered optimal for the RMP. Conversely, if one or more columns with negative reduced costs are identified, they are added to the RMP, providing a lower bound for the problem. If column generation stops and the current primal solution is fractional, the cut generation method separates lm-SRCs to tighten the lower bound. This process continues until no additional valid lm-SRCs are identified. A new iteration of the column-and-cut generation method is executed at this point. The column-and-cut generation algorithm continues until no new columns or rows are added to the RMP.

If the primal solution remains fractional, the algorithm explores the next node in the branch-and-bound tree until the optimal solution is obtained.

4.2. Branching Strategy

We employ two branching rules for the branching variable selection rule. The first branching rule pertains to auxiliary variables x_{ij}^k computed as $x_{ij}^k = \sum_{s \in \Omega^k} e_{ij}^s y_s^k$, where e_{ij}^s equals one if job j succeeds job i and they are consecutive in schedule S_s , and zero otherwise. It is worth noting that the integrality of the y -variable solution implies the integrality of the corresponding x -variable solution and vice versa. This relationship is proven in Chen and Powell (1999). If certain x_{ij}^k variables are fractional, we proceed to select a branching variable x_{hl}^v with the maximum integer infeasibility, specifically $x_{hl}^v = \operatorname{argmin}_{x_{ij}^k} |x_{ij}^k - 0.5|$.

Following Chen and Powell (1999), branching is implemented by updating the job-ordering restrictions (α, β) , rather than adding constraints to the RMP, as in Sadykov et al. (2021). The two branching rules do not alter the structure of the PSP^k; instead, they reduce the solution space by eliminating arcs from directed acyclic graphs. In our branch-and-bound approach, we combine two branching rules: first, we apply the q -variable branching rule, which, although not sufficient for imposing integrality constraints in the set-partitioning formulation, is effective at removing more arcs from directed acyclic graphs. When the q -variable solution—that is, q_{ij} , is integral—we use the x -variable branching rule to enforce integrality requirements.

We combine classical depth-first search and best-first search as the node-selection rule. If the current node in the branch-and-bound tree is not pruned, the depth-first search rule is applied, selecting one of the two child nodes of the current node as the next to be solved. Conversely, if the current node is pruned, the best-lower-bound rule is employed, choosing the active node in the branch-and-bound tree with the smallest lower bound as the next to be explored.

4.3. Acceleration Techniques

This section introduces state-of-the-art acceleration techniques, including primal heuristics, variable fixing via reduced costs, schedule enumeration, and multistage strong branching.

4.3.1. Primal Heuristic. To expedite convergence of the BPC algorithm, we employ the list-scheduling heuristic proposed by van den Akker et al. (1999) at the root node. The list scheduling heuristic is a widely used strategy in parallel machine scheduling.

4.3.2. Variable Fixing by Reduced Costs. Variable fixing by reduced cost is widely employed to expedite the BPC algorithm's solution process (Desaulniers et al. 2020). In our implementation, after solving the RMP using the column-and-cut generation method, we apply variable fixing by reduced cost, following a similar approach to Bulhões et al. (2020), which is equivalent to removing these arcs from the graph. For each machine k , we remove arcs from G^k that cannot be part of a path $s \in \Omega^k$, contributing to a solution that improves the incumbent solution.

Given an optimum dual solution (π, σ, μ) of the MP, with a lower bound LB as the value of the dual bound and an upper bound UB as the value of the incumbent solution, if the reduced cost $r_{a\min}^k$ of the schedule with the minimum reduced cost among all schedules containing the arc $a = (i, j)$ is larger than the primal-dual gap $UB - LB$, all schedules containing this arc can be fixed to zero—that is, $x_{ij}^k = 0$ —as per the node selection rule in Section 4.2. Implementing this rule removes arcs from the directed acyclic graph, thereby reducing job-ordering restrictions. Moreover, this process may trigger updates to upper bounds, as shown in Algorithm 2. Variable fixing by reduced cost operates at different nodes of the enumeration tree, because the pricing algorithm's structure, which operates on the resulting graph, remains unaffected. However, given the computational overhead of the exact backward labeling algorithm (see Section 3.5), the variable fixing procedure is not executed at every node. Instead, to strike a balance between computational cost and the benefits of arc reduction, it is triggered only when the primal-dual gap falls below 10%. For the sake of brevity, the detailed mathematical derivation and the calculation of $r_{a\min}^k$ are provided in Online Supplement EC.2.4.

4.3.3. Schedule Enumeration. A classical result in integer programming, often referred to as reduced-cost fixing, states that, given an upper bound UB and the optimal solution value LB of the linear programming (LP) relaxation of a binary integer program, any variable with a reduced cost greater than $UB - LB$ cannot assume a positive value in an optimal integer solution. Baldacci et al. (2008) were the first to demonstrate how this result can be

applied in the context of a column-generation method for the Capacitated VRP, leading to the development of a route enumeration method.

In our implementation, a dual solution (π, σ, μ) , a lower bound LB , and an upper bound UB are obtained after variable fixing by reduced costs. Then, the schedule enumeration method generates the subset $\bar{\Omega}^k$ such that $r_s^k \leq UB - LB$ for all schedules $s \in \bar{\Omega}^k$, $\forall k \in M$. The enumeration of schedules can be performed using a labeling algorithm, producing an SP formulation with reduced column sets, which is then given to a general integer programming (IP) solver. When the primal-dual gap is very small, the method works very well. Therefore, we have set this method to trigger only when the gap is less than 12%. To avoid the IP solver failing to solve or being too slow, we limit the size of the final schedule sets, $\sum_{k \in M} |\bar{\Omega}^k|$, to 30,000. Additionally, schedule enumeration can be embedded in the BPC algorithm and executed at different nodes of the enumeration tree. The forward labeling algorithm described in Section 3.4 can be easily tailored to perform schedule enumeration; details are omitted for brevity.

4.3.4. Multistage Strong Branching. Strong branching can significantly reduce the number of nodes requiring exploration within the enumeration tree (Dey et al. 2024). It involves rapidly assessing the influence of a set of potential branching candidates on the lower bounds of each subsequent child node. Subsequently, the optimal candidate is selected based on a predefined criterion.

In our implementation, we adopt a multistage strong branching procedure proposed by Pecin et al. (2017b) for our BPC. However, its improvement usually comes at the cost of increased time and memory per node during candidate branch selection. Therefore, we set the conditions for transforming multistage strong branching into general branching described in Section 4.2. Details are omitted for brevity; the reader is referred to Pecin et al. (2017b) for more information.

5. Numerical Experiments

In this section, we conduct an extensive experimental analysis of our BPC algorithm, using problem instances from existing literature and real-life data (Section 5.1). We first evaluate the performance of the BPC algorithm (Section 5.2) and compare it with state-of-the-art exact methods. Subsequently, we assess the role of each key component in our algorithm (Section 5.3). Finally, we evaluate the effectiveness of the key components of the BPC algorithm (Section 5.4).

5.1. Benchmark Instances and Setup

We consider instances from both the literature and real-world scenarios.

We use the data set introduced by Chen and Powell (1999) for literature-based instances. This data set is categorized into three sizes based on the number of machines (m) and jobs (n): *small-scale* instances with $m = 4$ and $n = 40, 60, 80$; *medium-scale* instances with $m = 8, 12$ and $n = 60, 80, 100$; and *large-scale* instances with $m = 16, 20$ and $n = 80, 100, 200$. The processing times (p_{jk}) are integers from a uniform distribution $U[1, 30]$, and weights (w_j) are generated using a uniform distribution $U[1, 100]$. For the consistency with other data sets, no additional consideration is given to job-machine incompatibilities, and N_k is set to N , for $k \in M$. We considered 15 groups, each comprising 20 instances, for a total of 300.

For real-world instances, data are collected from one warehouse in the largest retail warehouse (JD.com) in Beijing, China. More information about these real-world instances can be found in Online Supplement EC.4. The data set contains 60 instances, each with varying values of m and n . Therefore, we grouped these instances according to m and n : (1) $m \leq 12, 60 \leq n < 80$, including 1 instance; (2) $m \leq 12, 80 \leq n < 100$, including 5 instances; (3) $m \leq 12, 100 \leq n < 200$, including 5 instances; (4) $12 \leq m \leq 20, 80 \leq n < 100$, including 10 instances; (5) $12 \leq m \leq 20, 100 \leq n < 200$, including 17 instances; and (6) $12 \leq m \leq 20, n \geq 200$, including 22 instances. The resulting 60 real-world instances are categorized into six groups named according to the convention $RW - m - n$, where m ranges from 12 to 20 and n ranges from 60 to 200. For example, $RW - 12 - 60$ represents instances with 12–20 machines and 60–80 jobs.

The BPC algorithm is implemented in C# with ILOG CPLEX 12.8 as the LP solver. All tests are conducted on a computer with a 2.50-GHz Intel Core i9-12900H processor running Microsoft Windows 11 and equipped with 16.0 GB of RAM. Each BPC run was limited to one hour. The code of the proposed BPC algorithm, the instances, and detailed computational results are available online at Yu et al. (2026).

5.2. Results of the BPC Algorithm

The BPC algorithm proposed in this paper is compared with results from the following exact methods from the literature, which represent the state of the art in solving UPMSPPs.

- The branch-and-price algorithm for the UPMSP-WCT by Chen and Powell (1999), implemented in C and executed on a Silicon Graphics Iris Workstation with MIPS R4400 Processor. The LP solver is an older version of the existing ILOG CPLEX Optimizer. The algorithm proposed by Chen and Powell (1999) was executed on an older computational environment than ours, making it challenging to establish corresponding speed factors. To comprehensively demonstrate the performance of our BPC, we also executed the BPC in a variant adhering to the branch-and-price algorithm of Chen and Powell (1999), hereafter denoted as BP. Additionally, for completeness, we report the results presented by Chen and Powell (1999).

- Our implementation of the generic branch-cut-and-price algorithm for the UPMSP by Bulhões et al. (2020), denoted as BSSU. Note that Bulhões et al. (2020) did not address the UPMSP with the objective function $R \parallel \sum w_j C_j$ but focused on total weighted earliness and tardiness. Therefore, we implemented their approach to compare it with our BPC.

Results for the literature-based and real-world instances are given in Tables 2 and 3, respectively.

For each group of instances, the tables display the number of instances in the group. The tables for each algorithm provide the count of instances solved to optimality within the given time limit (“Opt”) and the average computing time in seconds (“Time (s)”). The column “Imp (%)” indicates the percentage improvement in BPC time relative to the algorithm time; positive values indicate better BPC performance. In the table, “t1” denotes a method that reached the time limit for all instances within a specific group.

As shown in Table 2, BPC solves 298 out of 300 instances, outperforming all other algorithms regarding the number of instances solved optimally, including larger instances with up to 200 jobs. In terms of computation time, it exhibits an average reduction of 79.78% compared with BP and 99.26% compared with BSSU. Key observations from this table include:

- BPC demonstrates exceptional performance when the ratio of jobs to machines (n/m) is relatively small. Conversely, as this ratio increases, the algorithm slows down. For instance, the average computation time for an instance with $m = 20$ and $n = 200$ is significantly lower than that for an instance with $m = 16$ and $n = 200$. Notably, six instances with $m = 16$ and $n = 200$ fail to reach optimality within 3,600 seconds.

- BP demonstrates high computational efficiency for small-scale instances but can only solve instances with up to 100 jobs. The algorithm’s inefficiency arises from the solving of PSP^k using dynamic programming, with worst-case complexity bounded by $O(n^2H)$ (Chen and Powell 1999), where $H = \sum_{j \in N} p_{jk}$. Hence, the algorithm slows down as H increases.

- BSSU solves 46 out of 300 instances, limited to instances with no more than 60 jobs. Similar to BP, the algorithm’s inefficiency in solving PSP^k using the general labeling algorithm (Pecin et al. 2017a) stems from limited dominance checks, resulting in slower performance as the completion time range increases (Sadykov et al. 2021).

Table 3 presents the computational results obtained on real-world instances. Because BP and BSSU are not directly applicable to instances with processing times represented by rational numbers, it is necessary to scale

Table 2. Computational Results on Instances from the Literature

Group		BPC			Chen and Powell (1999)			BP			BSSU		
m	n	Inst.	Opt	Time (s)	Opt	Time (s)	Imp (%)	Opt	Time (s)	Imp (%)	Opt	Time (s)	Imp (%)
4	40	20	20	3.79	20	51.15	92.59	20	8.43	55.04	20	327.00	98.84
4	60	20	20	43.94	20	623.35	92.95	20	78.57	44.07	1	3,343.22	98.69
4	80	20	20	240.42	0	t1	—	20	513.68	53.20	0	t1	—
8	60	20	20	7.47	20	87.97	91.51	20	45.64	83.63	15	2,056.32	99.64
8	80	20	20	27.89	20	487.41	94.28	20	155.00	82.01	0	t1	—
8	100	20	20	96.01	20	2,050.93	95.32	20	526.26	81.76	0	t1	—
12	60	20	20	4.21	20	48.02	91.23	20	44.13	90.46	10	3,245.14	99.87
12	80	20	20	12.66	20	203.34	93.77	20	163.14	92.24	0	t1	—
12	100	20	20	36.70	20	1,043.57	96.48	20	503.68	92.71	0	t1	—
16	80	20	20	9.63	20	206.42	95.33	20	97.69	90.14	0	t1	—
16	100	20	20	29.48	20	523.59	94.37	20	177.98	83.44	0	t1	—
16	200	20	18	564.29	0	t1	—	0	t1	—	0	t1	—
20	80	20	20	6.88	20	140.92	95.12	20	135.25	94.91	0	t1	—
20	100	20	20	20.83	20	362.8	94.26	20	321.59	93.52	0	t1	—
20	200	20	20	334.81	0	t1	—	0	t1	—	0	t1	—
Total		300	298		240			260			46		

Note. BPC, our BPC algorithm; BP and BSSU, our implementations of the Chen and Powell (1999) and Bulhões et al. (2020) algorithms; imp, percentage improvement in BPC time relative to the algorithm time; inst., instances; opt, instances solved to optimality; t1, a method that reached the time limit for all instances within a specific group.

Table 3. Computational Results on the Real-World Instances

Group	Inst.	BPC	
		Opt	Time (s)
RW-12-60	1	1	5.16
RW-12-80	5	5	19.45
RW-12-100	5	5	78.64
RW-20-80	10	10	11.82
RW-20-100	17	17	31.05
RW-20-200	22	11	1,424.51
Total	60	49	

Note. BPC, our BPC algorithm; inst., instances; opt, instances solved to optimality.

the processing times to integer values. However, this conversion results in the scaling of H and the range of completion times by hundreds. Consequently, BP and BSSU fail to solve any instance, as reported in Table 3. In contrast, the processing time required by rational numbers has little impact on the efficiency of our algorithm, and BPC successfully solves 49 out of 60 instances.

5.3. Analysis of the Role Played by the Different Components of BPC Algorithm

Tables 4 and 5 showcase the impacts of various components of BPC on instances derived from the literature.

Table 4, in the “Prim. heur.” section, presents the mean percentage gap (“Gap (%)”) between the values obtained by the primal heuristic and the optimal solution values, along with the average computational time in seconds (“Time (s)”). Under the “1st Col. Gener” heading, the table specifies the number of instances solved to optimality via column generation (“Opt”), the average number of columns generated (“Cols”), and the average computational time (“Time (s)”) at the root node of the enumeration tree. In the “1st lm-SRC” section, the table provides the number of instances solved to optimality through lm-SRC (“Opt”), the average number of generated lm-SRCs (“Cuts”), the number of instances where the lower bound is improved using lm-SRC cuts (“Imp Num”), and the average time spent separating them (“Time (s)”) at the root node. Table 5, under the heading “1st Vari. Fixi.”, provides details on the average number of removed arcs (“Arcs”) due to variable fixing, the number of updated dynamic upper bounds (“Num”), the proportion of dynamic shrink upper bound reduction attributable to variable fixing (“Reduce (%)”) at the root node, and the average computation time (“Time (s)”). In the “1st Sched. Enum.” section of Table 5, it reports the number of instances for which schedule enumeration is successfully performed (“Opt”), the number of generated schedules (“Cols”), and the average computing time

Table 4. Impact of the Different Components of the BPC Algorithm (First Part)

Group		Prim. heur.			1st Col. Gener.			1st lm-SRC			
m	n	Inst.	Gap (%)	Time (s)	Opt	Cols.	Time (s)	Opt	Cuts	Imp num	Time (s)
4	40	20	13.05	< 0.001	20	3,306.80	3.79	—	—	—	—
4	60	20	8.32	< 0.001	17	6,168.30	38.67	3	4.67	3	0.37
4	80	20	4.92	< 0.001	20	10,399.95	240.41	—	—	—	—
8	60	20	16.62	< 0.001	15	6,616.20	6.06	4	2.80	5	0.40
8	80	20	12.10	< 0.001	17	9,126.20	26.08	1	0.67	1	0.35
8	100	20	10.90	< 0.001	15	13,302.70	80.21	0	0.40	2	0.55
12	60	20	23.45	< 0.001	17	6,057.80	3.97	1	0.67	2	0.32
12	80	20	16.72	< 0.001	17	4,271.00	10.09	0	0.00	0	0.30
12	100	20	14.72	< 0.001	17	15,577.05	33.81	0	0.00	0	0.44
16	80	20	25.13	< 0.001	16	11,056.70	8.82	1	0.25	1	0.41
16	100	20	20.62	< 0.001	12	17,349.00	22.03	0	0.00	0	0.52
16	200	20	8.36	< 0.001	12	35,777.56	418.74	0	0.00	0	1.01
20	80	20	28.19	< 0.001	17	7,041.50	6.35	0	0.00	0	0.26
20	100	20	22.00	< 0.001	15	13,427.50	15.73	0	0.20	1	0.55
20	200	20	9.98	< 0.001	13	34,668.30	248.52	0	0.00	0	1.00
Total		300			240			10		15	

Note. Prim. heur., primal heuristic; Col. Gener., column generation; imp num, instances where the lower bound is improved; inst., instances; opt, instances solved to optimality.

Table 5. Impact of the Different Components of the BPC Algorithm (Second Part)

Group		1st Vari. Fixi.					1st Sched. Enum.			Stro. Bran.		
<i>m</i>	<i>n</i>	Inst.	Arcs	Num	Reduce (%)	Time (s)	Opt	Cols	Time (s)	Opt	Nodes	Time (s)
4	40	20	—	—	—	—	—	—	—	—	—	—
4	60	20	—	—	—	—	—	—	—	—	—	—
4	80	20	—	—	—	—	—	—	—	—	—	—
8	60	20	2,549.00	142.00	18.78	0.78	—	—	—	1	6.00	15.69
8	80	20	1,912.50	51.50	10.11	2.43	1	457	1.89	1	4.00	15.02
8	100	20	3,597.00	117.00	10.64	24.32	1	588	2.40	4	4.00	52.98
12	60	20	—	—	—	—	2	233.5	0.49	—	—	—
12	80	20	674.67	13.67	7.56	0.42	—	—	—	3	4.67	16.40
12	100	20	964.67	24.67	8.06	1.85	—	—	—	3	4.00	16.72
16	80	20	10,898.00	313.00	29.18	0.39	2	250	0.87	1	4.00	9.61
16	100	20	1,137.17	3.00	3.00	0.61	2	390	1.81	6	4.00	22.70
16	200	20	2,156.60	13.00	0.91	61.79	—	—	—	6	5.00	383.47
20	80	20	11,280.00	163.00	23.70	0.31	2	209.5	0.54	1	4.00	7.99
20	100	20	2,155.00	0.00	0.00	0.36	1	311	1.32	4	4.00	22.92
20	200	20	2,073.14	2.57	0.60	11.67	—	—	—	7	5.71	233.24
Total		300					11			37		

Note. Vari. Fixi., variable fixing; Sched. Enum., schedule enumeration; Stro. Bran., strong branching; inst., instances; num, updated dynamic upper bounds; opt, instances solved to optimality .

(“Time (s)”) at the root node. “Stro. Bran.” outlines the number of instances optimally solved via strong branching (“Opt”), “Nodes” the total number of explored nodes, and “Time (s)” the average computational time involved.

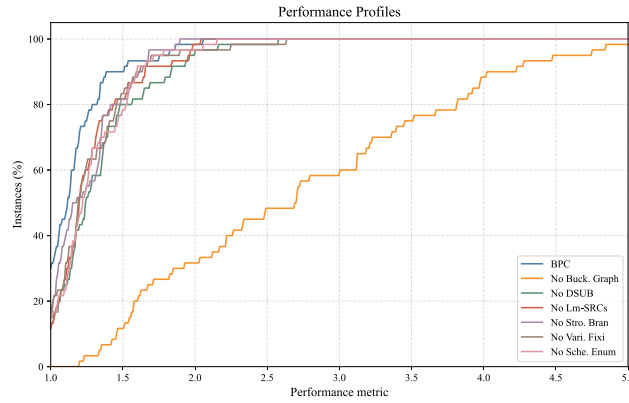
The results of Tables 4 and 5 can be summarized as follows:

- The primal heuristic generates an initial solution within a markedly short duration (substantially under 0.001 seconds) for instances of different scales. Although the average gap reaches 15.67%, the computation time is low enough to be acceptable for our BPC algorithm.
- The column generation method solves the majority of instances—that is, 240 out of 300 instances—emphasizing the crucial efficiency of column generation for the BPC algorithm.
- The separation of lm-SRCs, combined with the column-and-cut generation method, improves the lower bound for 15 out of the remaining 60 instances at the root node. For instance, in a case with 20 machines and 100 jobs, the lower bound increases from 22,699 to 22,713 (an improvement of 0.06%), significantly reducing computational time, as discussed in more detail in Section 5.4. Furthermore, the improved lower bound enables an additional 10 instances to be solved to optimality, eliminating the need for further branching.
- Variable fixing by reduced costs is particularly effective for medium-scale instances, enabling the rapid removal of numerous arcs and the updating of dynamic shrink upper bounds. For instance, for instances with 16 machines and 80 jobs, this approach removes an average of 10,898 arcs and updates 313 completion time upper bounds, achieving an average reduction rate of 29.18%.
- Schedule enumeration is successfully implemented on 11 of the remaining 50 instances at the root node, solving instances in less than three seconds, on average. For instance, for instances with 16 machines and 100 jobs, the approach only requires enumerating an average of 390 schedules and takes an average of 1.81 seconds to solve these instances to optimality.
- Multistage strong branching solves 37 of the remaining 39 instances. Benefiting from the strong branching technique, BPC only needs to explore an average of 4.49 nodes to obtain the optimal solution.

5.4. Effectiveness of the Main Components of the BPC Algorithm

To evaluate the contribution of the main components of the BPC algorithm, we examine eight variants, each with a specific component deactivated: the dynamic shrink upper bounds algorithm (“No DSUB”), the bucket graph-based labeling algorithm (“No Buck. Graph”), dominance rules (“No Domi. Rule”), lm-SRCs separation (“No Lm-SRC”), heuristic pricing (“No Heur. Pric.”), variable fixing by reduced costs (“No Vari. Fixi.”), schedule enumeration (“No Sche. Enum.”), and strong branching (“No Stro. Bran.”). Because of space constraints, the detailed performance metrics for these variants are provided in Tables EC.1 and EC.2 of the online supplement.

To more accurately assess the effectiveness of these components, we employ the *performance profile* concept to evaluate and contrast the performance of optimization algorithms. This approach is grounded in cumulative

Figure 2. (Color online) Performance Profiles of Different Methods Based on Computing Times (Small Scale Instances)

distribution functions corresponding to a specific performance metric. Here, the performance metric is defined as $t(x, p) / \min_x t(x, p)$, where $t(x, p)$ represents the computational time required by method x for solving instance p and $\min_x t(x, p)$ denotes the shortest computing time across all methods. For the given algorithm and metric, the cumulative distribution function $y = f(x)$ gives the proportion of instances for which the algorithm's performance equals x .

The performance profiles for different BPC versions in terms of their computing times for different scale instances are illustrated in Figures 2–4. For small-scale instances (Figure 2), the BPC algorithm slightly outperforms most methods; notably, the “No Buck. Graph” variant suffers a striking performance degradation. A distinctive shift in component importance emerges for medium-scale instances (Figure 3), where the “No Lm-SRCs” variant exhibits the most significant deterioration. For large-scale instances (Figure 4), the BPC algorithm remains superior in both the number of instances solved to optimality and the required computation time. Here, the “No DSUB” variant shows the most severe degradation. These results suggest that integrating a bucket graph-based labeling algorithm with dynamic shrink upper bounds substantially improves efficiency, particularly for large-scale instances. Furthermore, lm-SRCs are critical for accelerating medium-scale instances. Variable fixing and scheduled enumeration prove vital for medium- and large-scale instances, while strong branching plays a significant, albeit secondary, role in overall performance.

6. Conclusions

This paper presented a new branch-price-and-cut algorithm to address the Unrelated Parallel Machine Scheduling Problem to minimize the total weighted completion time. Our BPC algorithm uses several key components. Firstly, we represented feasible schedules of the problem as paths on directed acyclic graphs using Smith's rule. Subsequently, we proposed a labeling algorithm with a dynamic shrink upper bound algorithm, in which the

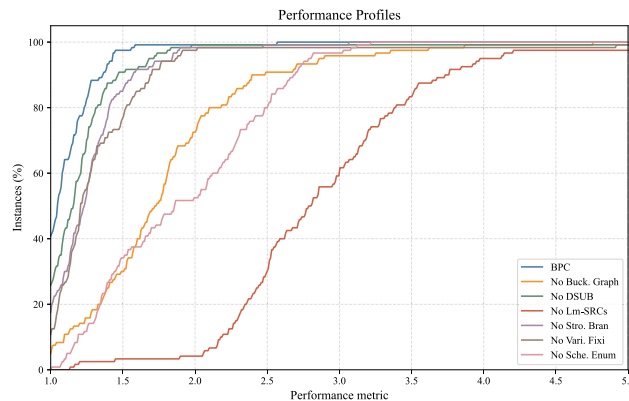
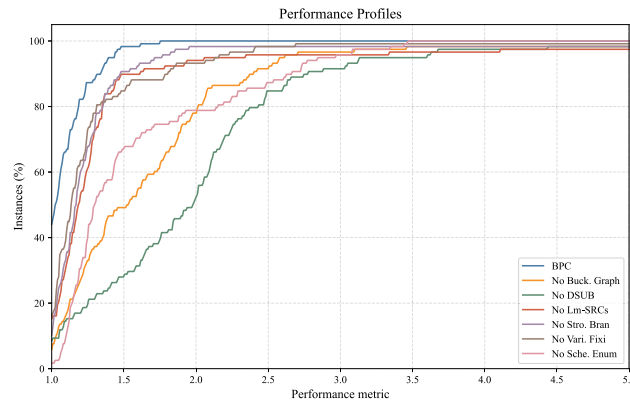
Figure 3. (Color online) Performance Profiles of Different Methods Based on Computing Times (Medium Scale Instances)

Figure 4. (Color online) Performance Profiles of Different Methods Based on Computing Times (Large Scale Instances)



upper bounds on job completion times are dynamically adjusted. The BPC algorithm also incorporates state-of-the-art technologies, such as a bucket graph-based labeling algorithm, limited-memory subset-row cuts, strong branching, and schedule enumeration.

Our extensive computational studies demonstrate the superiority of the new BPC algorithm. It can effectively handle instances featuring rational-number processing times, with up to 20 machines and 200 jobs for the first time. For benchmark instances with integer processing times sourced from the literature, our BPC algorithm successfully solves 298 out of 300 instances, with up to 20 machines and 200 jobs. This achievement surpasses all other algorithms in both the number of optimally solved instances and the time required to solve them.

There are three avenues for future research concerning these challenging optimization problems. Firstly, we aim to extend our BPC algorithm to address other variants of unrelated parallel machine scheduling problems. These include variants such as the makespan variant, the total weighted tardiness jobs variant, and the total weighted tardiness variant.

Additionally, our current problem model is static and does not account for dynamic events in real production environments, such as machine breakdowns and repairs (Kim and Kim 2021). Therefore, we plan to develop dynamic rescheduling methods to address this limitation, enabling us to respond swiftly and effectively to dynamic events. Furthermore, parallel machine scheduling typically occurs in environments with significant variability in job processing times. To tackle this challenge, which is often addressed through robust optimization (Song and Leus 2022), we aim to investigate solution algorithms to handle robust parallel machine scheduling problems.

Acknowledgments

The authors thank the anonymous reviewers and the associate editor for their valuable comments and thorough review of the paper. Yang Yu and Xiaolong Li contributed equally to the manuscript.

References

- Azizoglu M, Kirca O (1999) Scheduling jobs on unrelated parallel machines to minimize regular total cost functions. *IIE Trans.* 31(2):153–159.
- Baldacci R, Christofides N, Mingozzi A (2008) An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts. *Math. Program.* 115(2):351–385.
- Barnhart C, Johnson EL, Nemhauser GL, Savelsbergh MWP, Vance PH (1998) Branch-and-price: Column generation for solving huge integer programs. *Oper. Res.* 46(3):316–329.
- Bulhões T, Sadykov R, Subramanian A, Uchoa E (2020) On the exact solution of a large class of parallel machine scheduling problems. *J. Scheduling* 23(4):411–429.
- Che A, Zhang S, Wu X (2017) Energy-conscious unrelated parallel machine scheduling under time-of-use electricity tariffs. *J. Cleaner Production* 156:688–697.
- Chen ZL, Powell WB (1999) Solving parallel machine scheduling problems by column generation. *INFORMS J. Comput.* 11(1):78–94.
- Chen J, Chu C, Sahli A, Li K (2024) A branch-and-price algorithm for unrelated parallel machine scheduling with machine usage costs. *Eur. J. Oper. Res.* 316(3):856–872.
- Costa L, Contardo C, Desaulniers G (2019) Exact branch-price-and-cut algorithms for vehicle routing. *Transportation Sci.* 53(4):946–985.
- Desaulniers G, Gschwind T, Irnich S (2020) Variable fixing for two-arc sequences in branch-price-and-cut algorithms on path-based models. *Transportation Sci.* 54(5):1170–1188.
- Desrosiers J, Lübbecke M, Desaulniers G, Gauthier J (2024) Branch-and-price. Report, Les Cahiers du GERAD ISSN 711-2440, GERAD, Montréal, Québec.

- Detienne B, Dauzère-Pérès S, Yugma C (2011) Scheduling jobs on parallel machines to minimize a regular step total cost function. *J. Scheduling* 14(6):523–538.
- Dey SS, Dubey Y, Molinaro M, Shah P (2024) A theoretical and computational analysis of full strong-branching. *Math. Programming* 205(1–2): 303–336.
- Durasević M, Jakobović D (2023) Heuristic and metaheuristic methods for the parallel unrelated machines scheduling problem: A survey. *Artificial Intelligence Rev.* 56(4):3181–3289.
- Ekici A, Elyasi M, Özener OÖ, Sarıkaya MB (2019) An application of unrelated parallel machine scheduling with sequence-dependent setups at Vestel Electronics. *Comput. Oper. Res.* 111:130–140.
- Fukasawa R, Longo H, Lysgaard J, de Pragaço MP, Reis M, Uchoa E, Werneck RF (2006) Robust branch-and-cut-and-price for the capacitated vehicle routing problem. *Math. Programming* 106(3):491–511.
- Heath LS, Pemmaraju SV (1999) Stack and queue layouts of directed acyclic graphs: Part II. *SIAM J. Comput.* 28(5):1588–1626.
- Iori M, Locatelli A, Locatelli M (2023) A GRASP for a real-world scheduling problem with unrelated parallel print machines and sequence-dependent setup times. *Internat. J. Production Res.* 61(21):7367–7385.
- Jepsen M, Petersen B, Spoorendonk S, Pisinger D (2008) Subset-row inequalities applied to the vehicle-routing problem with time windows. *Oper. Res.* 56(2):497–511.
- Kim YI, Kim HJ (2021) Rescheduling of unrelated parallel machines with job-dependent setup times under forecasted machine breakdown. *Internat. J. Production Res.* 59(17):5236–5258.
- Kowalczyk D, Leus R (2018) A branch-and-price algorithm for parallel machine scheduling using ZDDs and generic branching. *INFORMS J. Comput.* 30(4):768–782.
- Lawler EL, Lenstra JK, Rinnooy Kan AHG, Shmoys DB (1993) Sequencing and scheduling: Algorithms and complexity. Graves SC, Rinnooy Kan AHG, Zipkin PH, eds. *Handbooks in Operations Research and Management Science* (Elsevier, Amsterdam), 445–522.
- Li K, Zhang X, Leung JYT, Yang SL (2016) Parallel machine scheduling problems in green manufacturing industry. *J. Manufacturing Systems* 38:98–106.
- Lin YK, Pfund ME, Fowler JW (2011) Heuristics for minimizing regular performance measures in unrelated parallel machine scheduling problems. *Comput. Oper. Res.* 38(6):901–916.
- Lübbecke ME, Desrosiers J (2005) Selected topics in column generation. *Oper. Res.* 53(6):1007–1023.
- Maecker S, Shen L, Mönch L (2023) Unrelated parallel machine scheduling with eligibility constraints and delivery times to minimize total weighted tardiness. *Comput. Oper. Res.* 149:105999.
- Pecin D, Contardo C, Desaulniers G, Uchoa E (2017a) New enhancements for the exact solution of the vehicle routing problem with time windows. *INFORMS J. Comput.* 29(3):489–502.
- Pecin D, Pessoa A, Poggi M, Uchoa E (2017b) Improved branch-cut-and-price for capacitated vehicle routing. *Math. Programming Comput.* 9(1):61–100.
- Pereira Lopes MJ, de Carvalho JMV (2007) A branch-and-price algorithm for scheduling parallel machines with sequence dependent setup times. *Eur. J. Oper. Res.* 176(3):1508–1527.
- Pessoa A, Sadykov R, Uchoa E, Vanderbeck F (2018) Automation and combination of linear-programming based stabilization techniques in column generation. *INFORMS J. Comput.* 30(2):339–360.
- Rodríguez FJ, Lozano M, Blum C, García-Martínez C (2013) An iterated greedy algorithm for the large-scale unrelated parallel machines scheduling problem. *Comput. Oper. Res.* 40(7):1829–1841.
- Rolim GA, Nagano MS, Prata BdA (2023) Formulations and an adaptive large neighborhood search for just-in-time scheduling of unrelated parallel machines with a common due window. *Comput. Oper. Res.* 153:106159.
- Sadykov R, Uchoa E, Pessoa A (2021) A bucket graph-based labeling algorithm with application to vehicle routing. *Transportation Sci.* 55(1):4–28.
- Sagnol G, Barner C, Borndörfer R, Grima M, Seeling M, Spies C, Werneck K (2018) Robust allocation of operating rooms: A cutting plane approach to handle lognormal case durations. *Eur. J. Oper. Res.* 271(2):420–435.
- Şen H, Bülbül K (2015) A Strong Preemptive Relaxation for Weighted Tardiness and Earliness/Tardiness Problems on Unrelated Parallel Machines. *INFORMS Journal on Computing*. 27(1):135–150.
- Shahvari O, Logendran R, Tavana M (2022) An efficient model-based branch-and-price algorithm for unrelated-parallel machine batching and scheduling problems. *J. Scheduling* 25(5):589–621.
- Shim SO, Kim YD (2007) Minimizing total tardiness in an unrelated parallel-machine scheduling problem. *J. Oper. Res. Soc.* 58(3):346–354.
- Smith WE (1956) Various optimizers for single-stage production. *Naval Res. Logist.* 3(1–2):59–66.
- Song G, Leus R (2022) Parallel machine scheduling under uncertainty: Models and exact algorithms. *INFORMS J. Comput.* 34(6):3059–3079.
- Uchoa E, Pessoa A, Moreno L (2024) Optimizing with Column Generation: Advanced branch-cut-and-price algorithms (Part I). Technical Report L-2024-3, Cadernos do LOGIS-UFF, Universidade Federal Fluminense, Engenharia de Produção, Niterói, RJ, Brazil, <https://optimizingwithcolumngeneration.github.io>.
- van den Akker JM, Hoogeveen JA, van de Velde SL (1999) Parallel machine scheduling by column generation. *Oper. Res.* 47(6):862–872.
- van der Aalst WMP (2016) *Process Mining: Data Science in Action*. 2nd ed. 2016 ed. (Springer, Berlin, Heidelberg).
- Wang T, Baldacci R, Lim A, Hu Q (2018) A branch-and-price algorithm for scheduling of deteriorating jobs and flexible periodic maintenance on a single machine. *Eur. J. Oper. Res.* 271(3):826–838.
- Xu X, Ma Y, Zhou Z, Zhao Y (2015) Customer order scheduling on unrelated parallel machines to minimize total completion time. *IEEE Trans. Automation Sci. Engng.* 12(1):244–257.
- Xu K, Li Y, Liu C, Liu X, Hao X, Gao J, Maropoulos PG (2020) Advanced data collection and analysis in data-driven manufacturing process. *Chinese J. Mech. Engng.* 33(1):43
- Yan X, Wang T, Shi X (2025) Optimal scheduling on unrelated parallel machines with combinatorial auction. *Ann. Oper. Res.* 344(2):937–963.
- Yu Y, Li X, Baldacci R, Wu Z, Sun W, Tang J, Zhu H (2026) An exact branch-price-and-cut algorithm for the unrelated parallel machine scheduling problem. <https://dx.doi.org/10.1287/ijoc.2024.0704.cd>, <https://github.com/INFORMSJoC/2024.0704>.
- Zhao L, Zhu H, Zhang M, Tang J, Wang Y (2026) Large-scale dynamic surgical scheduling under uncertainty by hierarchical reinforcement learning. *Internat. J. Production Res.* 64(2):719–750.

Copyright of INFORMS Journal on Computing is the property of INFORMS: Institute for Operations Research & the Management Sciences and its content may not be copied or emailed to multiple sites without the copyright holder's express written permission. Additionally, content may not be used with any artificial intelligence tools or machine learning technologies. However, users may print, download, or email articles for individual use.